Machine Learning
Semester 2026-III
Workshop No. 2 — Advanced Learning Models (Part A)

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the second workshop of the *Machine Learning* course project!

Your team has completed an initial exploratory analysis and baseline modeling phase. You now understand the problem domain, data characteristics, and baseline performance levels.

This workshop focuses on developing an enhanced predictive model using deep learning techniques appropriate for your assigned task. The model should achieve strong performance and produce meaningful learned representations. Architecture selection, training procedures, and optimization strategies should be tailored to your specific problem domain.

General Workshop Definition: Deep learning model development involves designing, training, and evaluating neural network architectures appropriate for the assigned task. Techniques such as transfer learning, regularization, and loss function adaptation should be applied strategically to address domain-specific challenges. In this workshop, teams implement, train, evaluate, and save their best-performing model using modern deep learning frameworks.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **Architecture Selection:** Choose a deep learning architecture appropriate for your problem domain (CNNs for images, RNNs/LSTMs for sequences, Transformers for NLP, value-based networks for RL, etc.).
- **Model Implementation:** Build, initialize, and validate the chosen architecture. Adapt pretrained models or initialize from scratch based on available data.
- **Training Strategy:** Design and implement a training procedure with appropriate loss functions, optimization, regularization, and learning rate schedules.
- **Handling Data Issues:** Apply techniques to address class imbalance, data distribution issues, or other domain-specific challenges identified in Workshop 1.
- **Training Monitoring:** Log all training runs, tracking loss, performance metrics, and learning dynamics across epochs.
- **Evaluation and Comparison:** Report comprehensive test-set metrics and compare against Workshop 1 baselines.
- **Model Checkpoint:** Save the best-performing model for use in subsequent workshops.

Architecture Selection and Transfer Learning

Choose an architecture appropriate to your problem domain:

- **Convolutional Neural Networks (CNNs):** For image-based or spatial data tasks. Common architectures: ResNet, EfficientNet, VGG, Inception. Pretrained on ImageNet are often available.
- **Recurrent Neural Networks (RNNs/LSTMs/GRUs):** For sequential data, time series, or temporal patterns. These architectures maintain hidden state across time steps.
- **Transformer Models:** For sequence-to-sequence tasks, NLP, and attention-based learning. Includes BERT, GPT, and domain-specific variants.
- **Value-Based Networks (DQN, Double DQN):** For reinforcement learning tasks where an agent learns action values.
- **Collaborative Filtering Networks:** For recommendation systems. Learns user and item embeddings jointly.
- **Custom Architectures:** Design a novel architecture tailored to your problem's specific structure and constraints.

Transfer Learning Strategy:

- When pretrained models exist for your domain (e.g., ImageNet-pretrained CNNs for images, BERT for NLP), leverage them: pretrained features often transfer well and reduce training time.
- Fine-tuning strategy: (1) Freeze pretrained layers, train only the task-specific head (5-10 epochs); (2) Unfreeze upper layers and continue training with lower learning rate (10-20 epochs).
- When no pretrained models are available, train from scratch with careful initialization and learning rate scheduling.
- Document your architecture choice and training procedure clearly.

Handling Data Issues:

- **Class Imbalance:** Apply class weighting in loss functions, use focal loss, or adjust sample weights during training.
- **Limited Training Data:** Use data augmentation, regularization (dropout, weight decay), and transfer learning to prevent overfitting.
- **Noisy Labels or Missing Values:** Document your approach to label cleaning or imputation. Consider robust loss functions.

Required Artifacts and Methodology

1. **Model Architecture Implementation:**
   - Select and implement your chosen architecture using `PyTorch`, `TensorFlow`, or equivalent framework.
   - Document the architecture: number of layers, dimensions, activation functions, and parameter counts.
   - Verify the model works by running a forward pass on a sample batch and checking output shapes.
2. **Training Procedure:**
   - Design a training loop: loss computation, backpropagation, optimization, validation.
   - Choose appropriate loss function for your task (cross-entropy for classification, MSE for regression, etc.).
   - Implement learning rate scheduling (exponential decay, cosine annealing, etc.).
   - Apply regularization (dropout, batch normalization, weight decay) to prevent overfitting.

- Train for a reasonable number of epochs, monitoring validation performance.

3. **Training Monitoring and Logging:**
   - Log training and validation loss per epoch.
   - Log relevant performance metrics (accuracy, F1, AUC-ROC for classification; MSE, MAE for regression; reward for RL, etc.).
   - Create learning curves showing loss and metrics over training.
   - Use `TensorBoard`, `Weights & Biases`, or equivalent tools for experiment tracking.
4. **Test-Set Evaluation and Comparison:**
   - Evaluate your trained model on the held-out test set using the same metrics as Workshop 1.
   - Report test-set performance and compare against Workshop 1 baselines.
   - If performance is significantly better than baselines, the model has learned domain-relevant patterns.
   - If performance is worse or only marginally better, investigate: data leakage? hyperparameter tuning needed? wrong architecture? insufficient training?
   - Create a results table: Workshop 1 baseline models vs. your Workshop 2 deep learning model.
5. **Model Checkpoint:**
   - Save the best-performing model checkpoint (highest validation metric) to `checkpoints/model_best.pt`.
   - Write a small test script that loads the checkpoint and verifies it produces predictions on new data.

Deliverables and Documentation

1. **Deep Learning Model Report (`PDF`):**
   - *Architecture Selection*: Why did you choose this architecture for your domain? What alternatives did you consider?
   - *Model Description*: Architecture details, parameter counts, input/output specifications.
   - *Training Protocol*: Loss function, optimizer, learning rate schedule, regularization techniques, hyperparameters.
   - *Training Results*: Learning curves, final training/validation metrics.
   - *Test-Set Evaluation*: Test-set performance, comparison against Workshop 1 baselines in a table.

- *Analysis*: What did the model learn? Where does it succeed or fail? How does performance compare to baselines?

2. **Code:**
   - `src/model.py` — Model architecture definition
   - `src/train.py` — Training script (data loading, training loop, validation)
   - `src/evaluate.py` — Test-set evaluation and metric computation
   - `src/test_model.py` — Script that loads saved checkpoint and runs inference
   - `checkpoints/model_best.pt` — Best model checkpoint
3. **Repository Structure:**
   - `src/` — all source code
   - `notebooks/` — Jupyter notebooks for experimentation
   - `checkpoints/` — saved model weights
   - `runs/` — training logs (TensorBoard/W&B)
   - `README.md` — instructions to reproduce training

Submission Requirements

- The `PDF` report must follow the naming convention: `groupN_workshop2.pdf`
- Submit through the designated course platform by the specified deadline
- Include the `GitHub` repository link with all code and model checkpoints
- All reported metric values must be reproducible by running the submitted code
- The model should achieve performance comparable to or better than Workshop 1 baselines

Grading Note: 40% of the workshop grade will be determined by an in-class quiz administered during the course. This quiz will assess your team's understanding of deep learning architecture selection, training procedures, and model performance analysis. The quiz emphasizes mastery of concepts and the ability to explain your design choices and results.

Important Guidelines and Considerations

- **Validation is Critical:** Never train on the test set. Use the validation set to monitor performance and choose when to stop training.
- **Architecture Matters:** The right architecture for your domain can dramatically improve performance. CNNs are not the solution for all problems.
- **Hyperparameter Tuning:** Try multiple learning rates, regularization strengths, and training procedures. Report your best result.
- **Reproducibility:** Document all hyperparameters, random seeds, and training procedures so results are reproducible.
- **Baseline Comparison:** Your deep learning model should substantially outperform Workshop 1 baselines. If it does not, investigate why before submitting.

Good luck with your deep learning journey. Choosing the right architecture and training procedure for your domain is an art that improves with practice. Your model is the foundation for everything that follows!